



Thapar Institute of Engineering & Technology

(Deemed to be University)

PATIALA

298965

3-16 Reasoning Please

Answer Sheet No.

Name. ARUNIMA JINDAL Roll No. 1024030907 Class/Group 2C64

Course No. UCS301 Course Name. DATA STRUCTURES

Name of the instructor. Dr. Ritesh Date 29 Sept 2025

INSTRUCTIONS

1. Write on both sides of the answer-book.
2. Do not remove or tear off any page from this answer book.
3. Candidates found guilty of unfair means or misconduct shall be dealt with as per Institute rules on unfair means.
4. Write your Roll No. on the Question Paper.
5. Write Page No. of each question (which you have attempted) in the table given below.

Evaluation:

Signature of the Invigilator

Q.No.	I	II	III	IV	V	VI	VII	VIII	IX	X	Total Marks
Page No.	1-3	4	7	8							
Marks	5 +	5 +	3+8	4.5 +1							

Total Marks in words: 3 1/2 7 1/2

Signature of the Examiner

Start Writing from here :-

1. (a)

• void func1 :

First for loop : $i = 1$ to $n \rightarrow$ runs ' n ' times,Time complexity = $O(n)$ For Second loop : $j = 1$ to $n \rightarrow$ runs ' n ' times,Time complexity $\rightarrow O(n)$ For Third loop : $k = 1$ to $10 \rightarrow$ constantTime complexity $\rightarrow O(1)$ $\therefore$ Time complexity of overall function = $O(n^2)$

as nested for loop is present.

(b) • func2 :

First for loop : $i = 0$ to $i \leq 2n \rightarrow$ runs ' $2n$ ' timesTime complexity $\approx O(n)$ For second loop : $j = 0$ to $j < i/3$, i goes from 0 to $2n$ $\therefore j = 0$ to $2n/3$ Time complexity $\approx O(n)$

for third loop : $k=0$ to $k < j^2$

max val of $j = 2n/3$

$$\therefore j^2 = \frac{4n^2}{9}$$

$$\therefore k=0 \text{ to } k < \frac{4n^2}{9}$$

$\therefore$ Time complexity $\approx O(n^2)$

$\therefore$ Nested for loop, Time complexity of func 2 = $O(n \times n \times n^2)$
 $= O(n^4)$

• func 3:

For first loop : $i=0$ to $i < n \rightarrow$ Time complexity = $O(n)$

For second loop : $j=i$ to $j < n$

$0 \rightarrow n, 1 \rightarrow n, 2 \rightarrow n, 3 \rightarrow n, \dots, n \rightarrow n.$

Loop runs $\rightarrow (n, n-1, n-2, n-3, \dots, 0)$

$$\begin{aligned} \sum (n + n-1 + \dots + 2 + 1 + 0) &= \sum (1 + 2 + 3 + \dots + n) \\ &= \sum \frac{n(n+1)}{2} \end{aligned}$$

$\therefore$ Loop runs approximately $\approx \frac{n^2 + n}{2}$

Time complexity $\approx O(n^2)$

for third loop : $k=j$ to $k < n$

$0 \rightarrow n, 1 \rightarrow n, \dots$

$$\text{loop runs} = \frac{n(n+1)}{2}$$

$\therefore$ Time complexity $\approx O(n^2)$

$\therefore$ Total time complexity = $O(n) \times O(n^2) \times O(n^2)$

$$= O(n^5)$$

1. (b) $\text{rain}[]$, $\text{maxLength} = N$,

For finding worst-case time complexity, we need to refer to the worst case scenario, which is that when the loop runs from i^{th} day to the last day that is, N , the strictly next greater rainfall should occur on the last day each time,

$\therefore$ the loop runs like

for (int $i = 1$; $i \leq N$; $i++$) {

~~$\text{arr}[i] < \text{arr}[i+1]$~~

for (int $j = 1$; $j \leq N$; $j++$) {

$\text{arr}[j] < \text{arr}[j+1]$;

return

for 1st iteration,

20 15 25 18 9 30

$j = 1$

$\text{arr}[0] < \text{arr}[1]$

$20 < 15 \times$

1/2

Algo: In each iteration, each value of $\text{arr}[]$ should ~~be~~ ^{be} compared with every other value that falls to its right and for the worst case scenario the rightmost element should be next greater or no next greater element.

This runs the inner loop each time to N times

$\therefore N$ elements must take $N \times N$ runs.

$\therefore$ Time complexity = $O(N^2)$.

Eg. $\text{rain}[] = \{20, 15, 25, 18, 9, 30\}$

Now, pick element 20,

compare with each following element:

$20 < 15 \times$

$20 < 25 \checkmark$

$\therefore$ Therefore next greater element exists,

thus, 2 days must be waited in this case for strictly greater rainfall.

Now pick element 15:

$15 < 25$

$\therefore$ one day

Now 25:

$25 < 18 \times$

$25 < 9 \times$

$25 < 30 \checkmark$

$\therefore$ 3 days.

Now 18:

$18 < 9 \times$

$18 < 30 \checkmark \therefore 2 \text{ days}$

Now 9:

$9 < 30 \checkmark$

$\therefore 1 \text{ day.}$

Now 30:

No such day. returns 0.

Therefore total no. of days one must wait = $2+1+3+2+1$
 $= 9 \text{ days.}$

2. (a) Postfix Evaluation

Expression: $60, 20, 2, /, +, 15, 5, *, -, 100, +$

Input	Stack	Output
60	60	Push 60 in stack
20	60, 20	Push 20 in stack
2	60, 20, 2	Push 2 in stack
/	60, 10	$20 \div 2 = 10$, $\therefore$ Stack is left with 60, 10.
+	70	Push 20 in stack
15	70, 15	Push 15 in stack
5	70, 15, 5	Push 5 in stack
*	70, 75	Multiply 15 and 5 = 75
-	-5	Subtract 70 & 75
100	-5, 100	Push 100 in stack
+	95	Add -5 and 100.

 $\therefore$ Find output = 95.

2. (b) Size of each Queue, Q_1 and $Q_2 = 5$.

$Q_1[5]$, $Q_2[5]$.

• $Q_1[5] = \{2, 5, 3, 6, 4\}$
 $\text{front}_1 = 0$ & $\text{rear}_1 = 4$

• $Q_2[5] = \{0\}$
 $\text{front}_2 = \text{rear}_2 = -1$

Before 1st iteration;

$Q_1 : \text{front}_1 = 0$, $\text{rear}_1 = 4$

$Q_2 : \text{front}_2 = -1$, $\text{rear}_2 = -1$

$k=1 \Rightarrow 1^{\text{st}}$ iteration:

$f = 2$

$Q_1.\text{pop}()$, $Q_1.\text{push}(3)$

$\therefore Q_1 = \{5, 3, 6, 4, 3\}$

$\text{front}_1 = 1$, $\text{rear}_1 = 0$

$Q_2 = \text{remains empty}$

$\therefore \text{front}_2 = \text{rear}_2 = -1$

$k=2 \Rightarrow 2^{\text{nd}}$ iteration:

$f = 5$

$\rightarrow Q_2.\text{push}(5)$, $Q_1.\text{pop}()$

$\therefore Q_1 = \{3, 6, 4, 3\}$

$Q_2 = \{5\}$

$\therefore \text{front}_1 = 2$, $\text{rear}_1 = 0$

$\text{front}_2 = \text{rear}_2 = 0$

$k=3 \Rightarrow 3^{\text{rd}}$ iteration:

~~$f = 3$~~

$\rightarrow Q_2.\text{push}(3)$, $Q_1.\text{pop}()$

$\therefore Q_2 = \{5, 3\}$

$\text{front}_2 = 0$, ~~front_2~~ $\text{rear}_2 = 1$

$Q_1 = \{6, 4, 3\}$

$\text{front}_1 = 3$, $\text{rear}_1 = 0$

$k=4 \Rightarrow 4^{\text{th}}$ iteration

$f=6$

$Q1.\text{pop}(), Q1.\text{push}(6)$

$\rightarrow Q1 = \{4, 3, 6\}$

$\text{front}1 = 4, \text{rear}1 = 1$

$Q2 = \{5, 3\}$

$\text{front}2 = 0, \text{rear}2 = 1$

$k=5 \Rightarrow 5^{\text{th}}$ iteration:

$f=4$

$Q1.\text{pop}(), Q1.\text{push}(5)$

$Q1 = \{3, 6, 5\}$

$\text{front}1 = 0, \text{rear}1 = 2$

$Q2 = \{5, 3\}$

$\text{front}2 = 0, \text{rear}2 = 1$

$\rightarrow$ change in output: when push $Q1 \rightarrow$ add element in front, $\text{pop}() \rightarrow$ removes from rear

$k=1 \Rightarrow 1^{\text{st}}$ iteration: $Q1 = \{3, 2, 5, 3, 6\}; \text{front}1 = 4, \text{rear}1 = 3$

$k=2 \Rightarrow 2^{\text{nd}}$ iteration: $Q2 = \{5\}; \text{front}2 = \text{rear}2 = 0$

~~$Q1$~~ $Q1 = \{3, 2, 5, 3\}; \text{front}1 = 4, \text{rear}1 = 2$

$k=3 \Rightarrow 3^{\text{rd}}$ iteration: $Q2 = \{3, 8\}; \text{front}2 = 4, \text{rear}2 = 0$

$Q1 = \{3, 2, 5\}; \text{front}1 = 4, \text{rear}1 = 1$

$k=4 \Rightarrow 4^{\text{th}}$

~~$Q1 = \{3, 2\}; \text{front}1 =$~~

$Q1 = \{6, 3, 2\}; \text{front}1 = 0, \text{rear}1 = 0$

~~$k=5$~~

$k=5:$

$Q1 = \{5, 6, 3\}$

~~$Q2 = f$~~

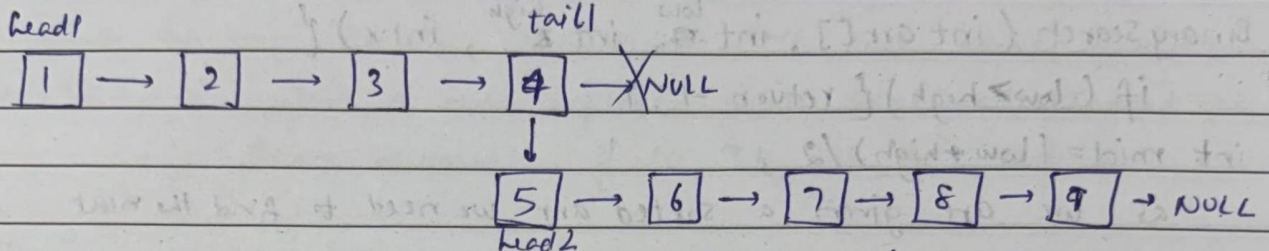
3. (a)

in Linked list 1 = $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$

LinkedList2 = $5 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 9$

head1 = 1

head2 = 5



point the next of last element of 1st linked list to the head of second linked list

Algorithm:

Step 1. Node * head1 = NULL → initially when list is empty.

for (int i = 0 ; i < m ; i++)

{

Node * newNode = new Node ;

if (head1 == NULL) {

head1 = newNode ;

}

else {

head1 → next = newNode ; }

}

Time Complexity = $O(m)$

3

Similarly take input and create linked list 2. (Sorted) → Time Complexity = $O(n)$

Step 2. Connect last element of linked list 1 to the first element of linked list 2, via, (let last element of linked list 1 be tail1)
tail1 → next = head2.

∴ Merged linked list 3 has the same head as that of the list which had smaller element first

∴ In this example,

head of linked list 3 = head1.

∴ Merged linked list = $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 9$.

3. (b) Sorted array $[0, 1, \dots, n-1]$

int = x.

~~we can perform this using Insertion Sorting~~

Using Binary Search:

Algorithm for Binary Search:

BinarySearch (int arr[], int ^{low}, int ^{high}, int x) {

if (low > high) { return -1; }

int mid = (low + high) / 2,

as we are given a sorted arr, we need to find the next greater element of x and then simply add it at that posⁿ and shift all other elements to the right by 1.

if (x > ~~arr[mid-1]~~ & & x < ~~arr[mid]~~)

insert x at mid and right shift all the elements in the array from mid+1.

arr[] = {10, 20, 30, 40, 50}, x = 25

low = 1

high = 5

mid = $6/2 = 3$

arr[mid] = 30

arr[mid-1] = 20

$x > 20$ & $x < 30$

$\therefore$ arr[] becomes = {10, 20, 25, 30, 40, 50}.

4. (a) yield [] = {45, 36, 78, 10, 12}

Selection Sort.

In selection sort, we divide our array in two parts $\rightarrow$ Sorted and Unsorted.

Initially array is assumed to be completely unsorted.
 step 1. ~~Find~~ ^{Let} the 'smallestIndex' at which the value of array element is minimum = i

smallestIndex = i (i starts from 0)

$\therefore$ Initially $arr[i] = arr[\text{smallestIndex}] = 45$

Now compare with all the other elements of the arr, and find the smallestIndex, which in this case comes out to be 3.
 $arr[3] = 10$, this is now replaced with $arr[\text{smallestIndex}] = 45$.

$\therefore$ Array becomes = $\{10, 36, 78, 45, 12\}$.

Similar process is to be done in each step:

Iteration 2:

$\{ \underbrace{10}_{\text{Sorted}}, \underbrace{36, 78, 45, 12}_{\text{Unsorted}} \}$ ~~$\{ 10, 36, 78, 45, 12 \}$~~

$\underbrace{10, 12}_{\text{Sorted}}, \underbrace{78, 45, 36}_{\text{Unsorted}}$

(~~78~~ 12 < 36)

Iteration 3:

$\underbrace{10, 12, 36}_{\text{Sorted}}, \underbrace{45, 78}_{\text{Unsorted}}$

(36 < 45)

Iteration 4:

$\underbrace{10, 12, 36, 45}_{\text{Sorted}}, 78$

(45 < 78)

$\therefore$ In 4 iterations, we can sort the given array using Selection Sort,

Final sorted array = $\{10, 12, 36, 45, 78\}$.

Time complexity = $O(n^2)$.

→ No selection sort is not always a stable sorting algorithm because of its higher Time Complexity (much more worse Time-complexity i.e., $O(n^2)$.)

4.5

4. (b)

1. ~~head = null~~

Assign first element.

1. ~~curr > MaxNode~~2. ~~curr < minNode~~3. ~~prev → next~~4. ~~curr → next~~5. ~~minNode → next = NULL~~6. ~~maxPrev = curr.~~

7.

8. ~~minNode → next = head~~9. ~~minPrev = curr~~10. ~~minNode = temp~~